Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №4
По дисциплине: «Обработка изображений в интеллектуальных системах»
Тема: «Трекинг множественных объектов»

Выполнила:
Студент 4 курса
Группы ИИ-26
Карпович К.А.
Проверила:
Андренко К.В.

Брест 2026

Цель работы: исследовать применение алгоритмов трекинга на базе обученной сети-детектора объектов.

Общее задание

1. Используя сеть-детектор, обученный в ЛР 3, реализовать логику для отслеживания множественных объектов, используя библиотеку Ultralytics YOLO;
2. Применять алгоритмы BoT-Sort и ByteTrack (задействовать соответствующие конфигурационные файлы);
3. Исследовать изменения параметров в конфигурационных файлах и их влияние на качество трекинга;
4. В качестве исходных видеоматериалов для экспериментов использовать видео-ролики из сети (например, из YouTube), содержащие множественные объекты классов из ЛР 3;
5. Оформить отчет по выполненной работе, залить исходный код и отчет в соответствующий репозиторий на github

В качестве детектора использована модель YOLO11m, дообученная в ЛР 3 на наборе Rock-Paper-Scissors (классы Rock, Paper, Scissors; mAP@0.5 = 0,941 на тестовой выборке). Трекинг выполнялся методом model.track() библиотеки Ultralytics. Эксперименты проводились в Google Colab на GPU Tesla T4.

Исходное видео. [How to Play Rock, Paper, Scissors], длительность [1 мин 10 с], 4151 кадр. В кадре одновременно находится до 3 рук (в среднем 1,42), камера статична, жесты показываются быстро, руки перекрывают друг друга и выходят из кадра.

Реализация. Скрипт для каждого эксперимента создаёт конфигурационный файл на основе стандартных bytetrack.yaml и botsort.yaml, изменяя один параметр, запускает трекинг, записывает видео с рамками, идентификаторами и траекториями и считает метрики. Порог уверенности детектора conf = 0,1, чтобы слабые детекции доходили до второй стадии ассоциации.

Так как эталонной разметки треков нет, использованы косвенные метрики:

- **unique_ids -** число выданных идентификаторов;
- **max_simultaneous -** максимальное число объектов в одном кадре;
- **excess_ids** = unique_ids − max_simultaneous **-** число «лишних» идентификаторов (потери и перепутывания треков);
- **mean/median_track_len -** средняя и медианная длина трека в кадрах;
- **short_tracks -** число треков короче 15 кадров (обрывки и ложные срабатывания);
- **fps -** скорость обработки.

Чем меньше лишних ID и коротких треков и чем длиннее треки, тем лучше трекинг.

Таблица 1 - Исследуемые параметры

Параметр	Назначение	Значения
track_high_thresh / new_track_thresh	порог первой стадии ассоциации / создания нового трека	0,25 (по умолч.); 0,5
track_low_thresh	порог второй стадии (слабые детекции)	0,1 (по умолч.); 0,3
track_buffer	сколько кадров хранится потерянный трек	30 (по умолч.); 10; 90
match_thresh	порог стоимости сопоставления (1 − IoU)	0,8 (по умолч.); 0,6; 0,95
gmc_method (BoT-SORT)	компенсация движения камеры	sparseOptFlow (по умолч.); none; orb
with_reid (BoT-SORT)	использование ReID-признаков	False (по умолч.); True

Код программы:

```
import argparse
import csv
import json
import time
from collections import defaultdict, deque
from pathlib import Path
```

```
import cv2
import yaml
from ultralytics import YOLO
from ultralytics.utils import ROOT

EXPERIMENTS = {
    "bytetrack_default": ("bytetrack", {}),
    "bytetrack_high_thresh": ("bytetrack", {"track_high_thresh": 0.5, "new_track_thresh": 0.5}),
    "bytetrack_low_thresh": ("bytetrack", {"track_low_thresh": 0.3}),
    "bytetrack_buffer_10": ("bytetrack", {"track_buffer": 10}),
    "bytetrack_buffer_90": ("bytetrack", {"track_buffer": 90}),
    "bytetrack_match_0.6": ("bytetrack", {"match_thresh": 0.6}),
    "bytetrack_match_0.95": ("bytetrack", {"match_thresh": 0.95}),
    "botsort_default": ("botsort", {}),
    "botsort_high_thresh": ("botsort", {"track_high_thresh": 0.5, "new_track_thresh": 0.5}),
    "botsort_low_thresh": ("botsort", {"track_low_thresh": 0.3}),
    "botsort_buffer_10": ("botsort", {"track_buffer": 10}),
    "botsort_buffer_90": ("botsort", {"track_buffer": 90}),
    "botsort_match_0.6": ("botsort", {"match_thresh": 0.6}),
    "botsort_match_0.95": ("botsort", {"match_thresh": 0.95}),
    "botsort_gmc_none": ("botsort", {"gmc_method": "none"}),
    "botsort_gmc_orb": ("botsort", {"gmc_method": "orb"}),
    "botsort_reid": ("botsort", {"with_reid": True}),
}


def color_for(track_id):
    return (int(track_id) * 37 % 200 + 55, int(track_id) * 91 % 200 + 55, int(track_id) * 53 % 200 + 55)


def build_config(name, tracker_type, overrides, config_dir):
    with open(ROOT / "cfg" / "trackers" / f"{tracker_type}.yaml", encoding="utf-8") as f:
        cfg = yaml.safe_load(f)
    cfg.update(overrides)
    path = config_dir / f"{name}.yaml"
    with open(path, "w", encoding="utf-8") as f:
        yaml.safe_dump(cfg, f, allow_unicode=True, sort_keys=False)
    return path


def video_info(source):
    cap = cv2.VideoCapture(str(source))
    fps = cap.get(cv2.CAP_PROP_FPS) or 25.0
    width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
    height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
    total = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
    cap.release()
    return fps, width, height, total


def run_experiment(weights, source, cfg_path, video_path, args):
    model = YOLO(weights)
    fps, width, height, total = video_info(source)
    stride = max(args.vid_stride, 1)
    total = total // stride
    if args.max_frames:
        total = min(total, args.max_frames)
    writer = None
    if video_path is not None:
        writer = cv2.VideoWriter(str(video_path), cv2.VideoWriter_fourcc(*"mp4v"), fps / stride, (width, height))

    lifetimes = defaultdict(int)
    trails = defaultdict(lambda: deque(maxlen=args.trail))
    active_per_frame = []
    frames = 0
    start = time.time()

    results = model.track(
        source=str(source),
        tracker=str(cfg_path),
        conf=args.conf,
        iou=args.iou,
        imgsz=args.imgsz,
        device=args.device,
        vid_stride=stride,
        stream=True,
        persist=False,
        verbose=False,
    )

    for result in results:
        frames += 1
        boxes = result.boxes
        ids = []
        if boxes is not None and boxes.id is not None:
            ids = boxes.id.int().cpu().tolist()
            xyxy = boxes.xyxy.cpu().numpy()
            for track_id, (x1, y1, x2, y2) in zip(ids, xyxy):
                lifetimes[track_id] += 1
                trails[track_id].append((int((x1 + x2) / 2), int(y2)))
        active_per_frame.append(len(ids))

        if writer is not None:
            frame = result.plot()
            for track_id in ids:
                points = list(trails[track_id])
```

```
                for a, b in zip(points[:-1], points[1:]):
                    cv2.line(frame, a, b, color_for(track_id), 2)
            writer.write(frame)

        if frames % args.progress == 0:
            speed = frames / max(time.time() - start, 1e-9)
            left = (total - frames) / max(speed, 1e-9)
            print(f"    кадр {frames}/{total}  {speed:.1f} кадр/с  осталось ~{left:.0f} с", flush=True)

        if args.max_frames and frames >= args.max_frames:
            break

    elapsed = max(time.time() - start, 1e-9)
    if writer is not None:
        writer.release()

    lengths = sorted(lifetimes.values())
    unique_ids = len(lengths)
    max_simultaneous = max(active_per_frame) if active_per_frame else 0
    median_len = lengths[len(lengths) // 2] if lengths else 0
    return {
        "frames": frames,
        "unique_ids": unique_ids,
        "max_simultaneous": max_simultaneous,
        "excess_ids": max(unique_ids - max_simultaneous, 0),
        "mean_active": round(sum(active_per_frame) / max(frames, 1), 2),
        "mean_track_len": round(sum(lengths) / max(unique_ids, 1), 1),
        "median_track_len": median_len,
        "short_tracks": sum(1 for l in lengths if l < args.short_len),
        "fps": round(frames / elapsed, 1),
    }


def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--weights", required=True)
    parser.add_argument("--sources", nargs="+", required=True)
    parser.add_argument("--out", default="tracking_results")
    parser.add_argument("--only", nargs="*", default=None)
    parser.add_argument("--conf", type=float, default=0.1)
    parser.add_argument("--iou", type=float, default=0.7)
    parser.add_argument("--imgsz", type=int, default=640)
    parser.add_argument("--device", default=None)
    parser.add_argument("--trail", type=int, default=40)
    parser.add_argument("--short-len", type=int, default=15)
    parser.add_argument("--vid-stride", type=int, default=1)
    parser.add_argument("--max-frames", type=int, default=0)
    parser.add_argument("--progress", type=int, default=20)
    parser.add_argument("--no-video", action="store_true")
    args = parser.parse_args()

    out_dir = Path(args.out)
    config_dir = out_dir / "configs"
    video_dir = out_dir / "videos"
    config_dir.mkdir(parents=True, exist_ok=True)
    video_dir.mkdir(parents=True, exist_ok=True)

    names = args.only if args.only else list(EXPERIMENTS)
    rows = []

    for source in args.sources:
        source_name = Path(source).stem
        for name in names:
            tracker_type, overrides = EXPERIMENTS[name]
            cfg_path = build_config(name, tracker_type, overrides, config_dir)
            print(f"[СТАРТ] {source_name} / {name}", flush=True)
            video_path = None if args.no_video else video_dir / f"{source_name}__{name}.mp4"
            try:
                stats = run_experiment(args.weights, source, cfg_path, video_path, args)
            except Exception as error:
                print(f"[ПРОПУСК] {source_name} / {name}: {error}")
                continue
            row = {
                "video": source_name,
                "experiment": name,
                "tracker": tracker_type,
                "params": json.dumps(overrides, ensure_ascii=False),
                **stats,
            }
            rows.append(row)
            print(
                f"{source_name:20s} {name:24s} ids={stats['unique_ids']:4d} "
                f"max_sim={stats['max_simultaneous']:3d} excess={stats['excess_ids']:4d} "
                f"mean_len={stats['mean_track_len']:7.1f} short={stats['short_tracks']:4d} "
                f"fps={stats['fps']:6.1f}"
            )

    if rows:
        csv_path = out_dir / "results.csv"
        with open(csv_path, "w", newline="", encoding="utf-8-sig") as f:
            writer = csv.DictWriter(f, fieldnames=list(rows[0].keys()), delimiter=";")
            writer.writeheader()
            writer.writerows(rows)
        print("Таблица сохранена:", csv_path)

if __name__ == "__main__":
    main()
```

Результаты обучения:

Таблица 2 - Результаты экспериментов (4151 кадр)

experiment	tracker	params	frames	unique_ids	max_simultaneous	excess_ids	mean_active	mean_track_len	median_track_len	short_tracks	fps
bytetrack_default	bytetrack	{}	4151	50	3	47	1.42	117.8	18	25	34.3
bytetrack_high_thresh	bytetrack	{"track_high_thresh": 0.5, "new_track_thresh": 0.5}	4151	34	3	31	1.36	166.2	58	12	34.5
bytetrack_low_thresh	bytetrack	{"track_low_thresh": 0.3}	4151	52	3	49	1.38	110.0	13	27	34.3
bytetrack_buffer_10	bytetrack	{"track_buffer": 10}	4151	53	3	50	1.42	111.0	18	26	33.8
bytetrack_buffer_90	bytetrack	{"track_buffer": 90}	4151	50	3	47	1.42	117.8	14	26	33.7
bytetrack_match_0.6	bytetrack	{"match_thresh": 0.6}	4151	116	3	113	1.34	48.0	3	79	34.2
bytetrack_match_0.95	bytetrack	{"match_thresh": 0.95}	4151	39	3	36	1.43	152.4	28	18	34.4
botsort_default	botsort	{}	4151	48	3	45	1.42	123.1	14	25	28.2
botsort_high_thresh	botsort	{"track_high_thresh": 0.5, "new_track_thresh": 0.5}	4151	32	3	29	1.36	176.6	69	12	28.1
botsort_low_thresh	botsort	{"track_low_thresh": 0.3}	4151	53	3	50	1.38	108.1	9	28	27.7
botsort_buffer_10	botsort	{"track_buffer": 10}	4151	55	3	52	1.42	107.2	13	29	27.7
botsort_buffer_90	botsort	{"track_buffer": 90}	4151	46	3	43	1.42	128.5	21	23	27.8
botsort_match_0.6	botsort	{"match_thresh": 0.6}	4151	106	3	103	1.35	52.9	3	77	28.1
botsort_match_0.95	botsort	{"match_thresh": 0.95}	4151	41	3	38	1.43	145.1	28	19	28.3
botsort_gmc_none	botsort	{"gmc_method": "none"}	4151	47	3	44	1.42	125.7	13	25	34.6
botsort_gmc_orb	botsort	{"gmc_method": "orb"}	4151	49	3	46	1.42	120.5	14	25	32.2
botsort_reid	botsort	{"with_reid": true}	4151	51	4	47	1.43	116.7	12	29	27.4

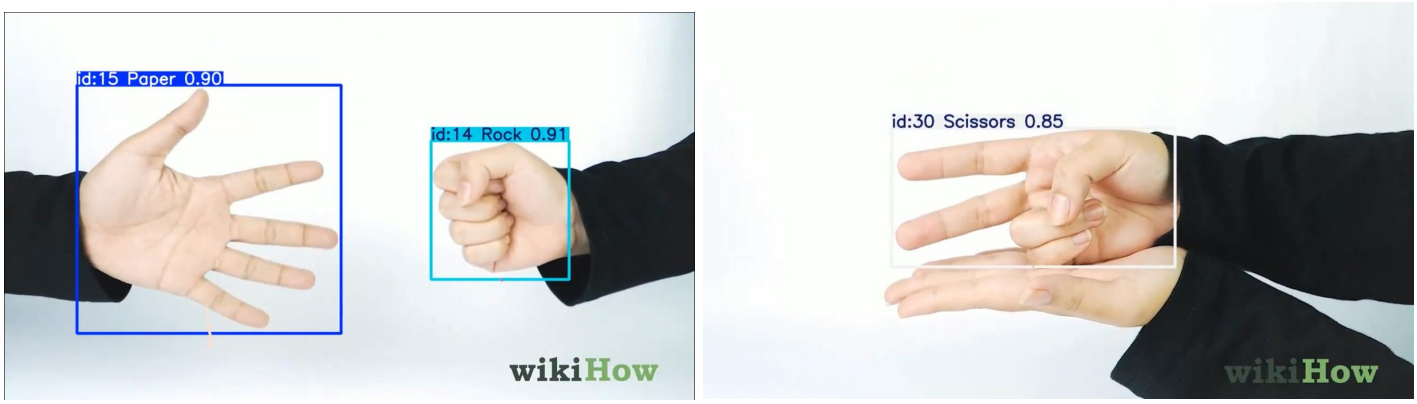


Рис. 1 - кадры из видео: ByteTrack по умолчанию и с high_thresh 0,5

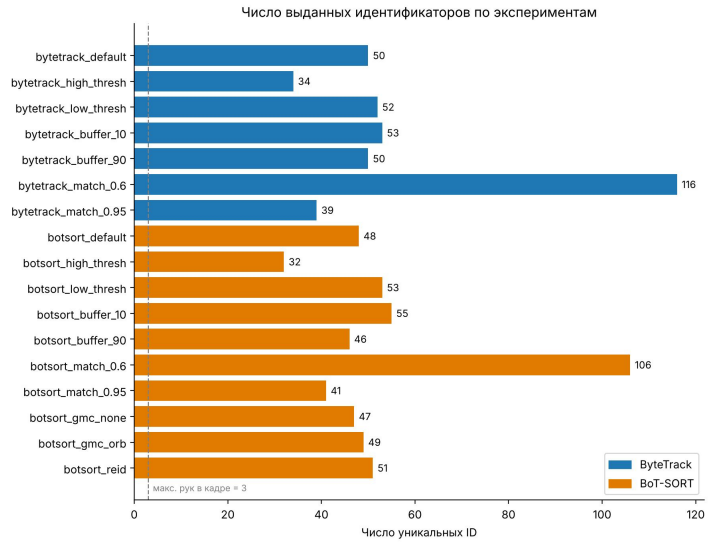


Рис. 2 - график: число уникальных ID по экспериментам

Анализ результатов

- **Пороги уверенности.** Повышение track_high_thresh и new_track_thresh до 0,5 дало лучший результат: число ID снизилось на треть (ByteTrack 50→34, BoT-SORT 48→32), коротких треков стало вдвое меньше, медианная длина трека выросла с 14–18 до 58–69 кадров. Низкие пороги при conf = 0,1 порождают много ложных коротких треков. Повышение track_low_thresh до 0,3 результат слегка ухудшило.
- **match_thresh.** Это порог на стоимость сопоставления (1 − IoU), поэтому чем он выше, тем меньшее перекрытие рамок допускается. Значение 0,6 резко ухудшило трекинг (106–116 ID): при быстром движении рук рамки в соседних кадрах перекрываются слабо, и трекер теряет объект. Значение 0,95 улучшило результат (39–41 ID).
- **track_buffer.** Влияние слабое: увеличение до 90 почти не изменило результат у ByteTrack (50 ID) и немного улучшило у BoT-SORT (46), уменьшение до 10 слегка ухудшило. Значит, обрывы треков вызваны не долгим отсутствием руки, а быстрым движением и неустойчивым обнаружением между жестами.
- **BoT-SORT и ByteTrack.** BoT-SORT дал чуть лучшие показатели (32 против 34 ID при лучших параметрах), но работает медленнее (≈28 против ≈34 кадр/с).
- **Компенсация движения камеры.** Камера в видео статична, поэтому gmc_method = none не ухудшил качество (47 ID против 48), а скорость выросла с 28,2 до 34,6 кадр/с.
- **ReID.** Включение with_reid не улучшило результат (51 ID) и замедлило обработку: для коротких жестов без выраженного внешнего вида ReID-признаки малоинформативны.

Вывод: в ходе работы реализован трекинг множественных объектов на основе детектора YOLO11m из ЛР 3 с алгоритмами ByteTrack и BoT-SORT (библиотека Ultralytics). Проведено 17 экспериментов на видео из сети (4151 кадр, до 3 рук в кадре). Наибольшее влияние на качество оказали порог track_high_thresh/new_track_thresh (0,5 сократил число ID на треть и вдвое число коротких треков) и match_thresh (0,95 улучшает, 0,6 резко ухудшает трекинг). Параметр track_buffer, компенсация движения камеры и ReID влияют слабо. Лучший результат показал BoT-SORT с track_high_thresh = new_track_thresh = 0,5 (32 ID, медианная длина трека 69 кадров). Число ID (32) всё же заметно больше числа рук (3): основная причина - неустойчивое обнаружение руки детектором во время смены жеста и при быстром движении. Для оценки качества использованы косвенные метрики, так как эталонной разметки треков нет.